

Descripción General del Sistema de Asignación de Horarios de Profesores

El Sistema de Asignación de Horarios de Profesores:

Es una plataforma tecnológica desarrollada para optimizar y automatizar la planificación académica en instituciones educativas. Funciona en un entorno controlado donde interactúan dos actores principales: Administrador y Profesor.

El Administrador tiene la responsabilidad de gestionar el registro de profesores, la asignación de materias, la planificación de horarios y la asignación de grupos estudiantiles. Por su parte, el Profesor puede consultar su horario, las materias que impartirá y los grupos asignados a través de un acceso personalizado.

Desde el punto de vista técnico, el sistema integra algoritmos especializados para mejorar la eficiencia y evitar errores comunes como sobrecargas de trabajo o conflictos de horarios. Se destaca el uso del algoritmo de Gale-Shapley para emparejamiento estable entre profesores y materias según su especialización, y la programación lógica con restricciones, utilizada para resolver conflictos de disponibilidad de aulas y horarios.

El sistema cuenta con un modelo de control de acceso basado en roles, garantizando que cada usuario acceda solo a las funciones correspondientes a su perfil, asegurando así la seguridad y confidencialidad de la información académica.

Además, incluye un módulo especializado para la generación automática de reportes académicos y administrativos, mediante la clase *GeneradorReportes*, que organiza la información proveniente de módulos como *RegistroEventos* y *RegistroVisitas*.

Gracias a su diseño modular y escalable, este sistema está preparado para adaptarse a futuras necesidades, como el aumento de matrícula, nuevos docentes o mayor variedad de asignaturas, sin comprometer su estabilidad.

En conjunto, este sistema contribuye a una gestión académica más organizada, transparente y eficiente, maximizando el aprovechamiento de los recursos institucionales.

Arquitectura y Funcionalidad

El sistema ha sido desarrollado bajo una arquitectura modular distribuida en capas, lo que permite una clara separación de responsabilidades, facilitando el mantenimiento, la escalabilidad y la integración entre componentes. Esta arquitectura está compuesta por diversas capas funcionales que trabajan en conjunto para garantizar una asignación eficiente de materias y horarios a los profesores, considerando su disponibilidad, especialidad y evitando conflictos de agenda.

En la capa de presentación, los elementos como *View*, *Componentes* y *Frontend* conforman la interfaz gráfica, la cual permite a los usuarios principalmente profesores y administradores visualizar horarios, materias asignadas y recibir notificaciones del sistema. Esta interfaz actúa como el punto de interacción principal entre el sistema y sus usuarios.

La capa de lógica de negocio está centrada en la clase *sistemaAsignación*, que gestiona el proceso principal de asignación de materias y horarios. Este módulo se apoya en los *Controllers* y *Servicios* para consultar y actualizar información sobre materias, profesores y horarios. Además, el *GeneradorReportes* es responsable de emitir informes detallados sobre asignaciones realizadas y actividades registradas en el sistema.

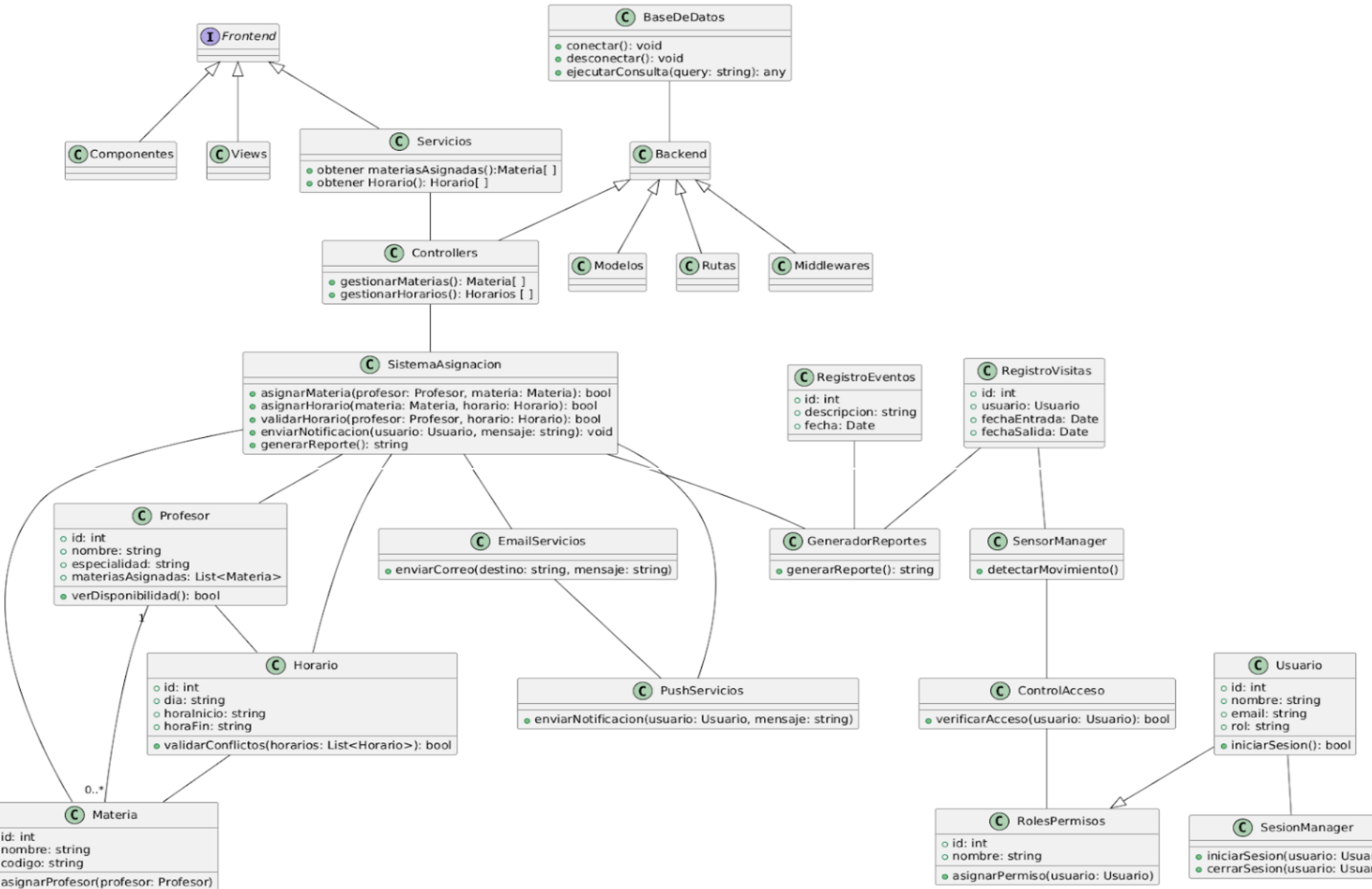
La capa de comunicación se encarga de mantener informados a los usuarios sobre los cambios y eventos importantes. Para ello, se utilizan los componentes *EmailServicios* y *PushServicios*, los cuales permiten enviar correos electrónicos y notificaciones en tiempo real de manera automatizada.

En cuanto a la capa de seguridad, esta se conforma por *SesionManager*, *RolesPermisos* y *ControlAcceso*, los cuales aseguran que cada usuario pueda acceder únicamente a las funcionalidades autorizadas según su rol dentro del sistema. Estas herramientas permiten una gestión robusta de sesiones y privilegios.

La persistencia de los datos se maneja a través de componentes como *BaseDeDatos*, *Modelos*, *Rutas* y *Middlewares*, los cuales se encargan de establecer la conexión con la base de datos, realizar consultas, manejar rutas de acceso a datos y aplicar validaciones necesarias para garantizar la integridad y seguridad de la información.

Por otro lado, se incorpora una capa adicional enfocada en la supervisión de accesos físicos y digitales mediante el *SensorManager*, que detecta movimiento o presencia, y los módulos *RegistroEventos* y *RegistroVisitas*, que documentan entradas, salidas y actividades relevantes, fortaleciendo así el control y la seguridad del sistema.

Figura 1. Diagrama de clases.



Patrón Modelo-Vista-Controlador (MVC):

Un patrón que separa la aplicación en tres componentes interconectados: Modelo (datos y lógica de negocios), Vista (interfaz de usuario) y Controlador (maneja la entrada del usuario y actualiza el modelo y la vista). Componentes clave: Modelo, Vista, Controlador.

Implementación en el Sistema

El sistema de asignación de horarios utilizará el patrón MVC para separar la lógica de negocio de la presentación y la gestión de eventos. El controlador se encarga de la comunicación entre la vista y el modelo, evitando dependencias directas entre ellos.

Beneficios

- Modularidad: Separa la lógica de negocio de la interfaz de usuario.
- Facilita el mantenimiento: Cambios en la vista no afectan al modelo y viceversa.
- Reutilización: Permite reutilizar el modelo en diferentes interfaces (Web, móvil, escritorio).

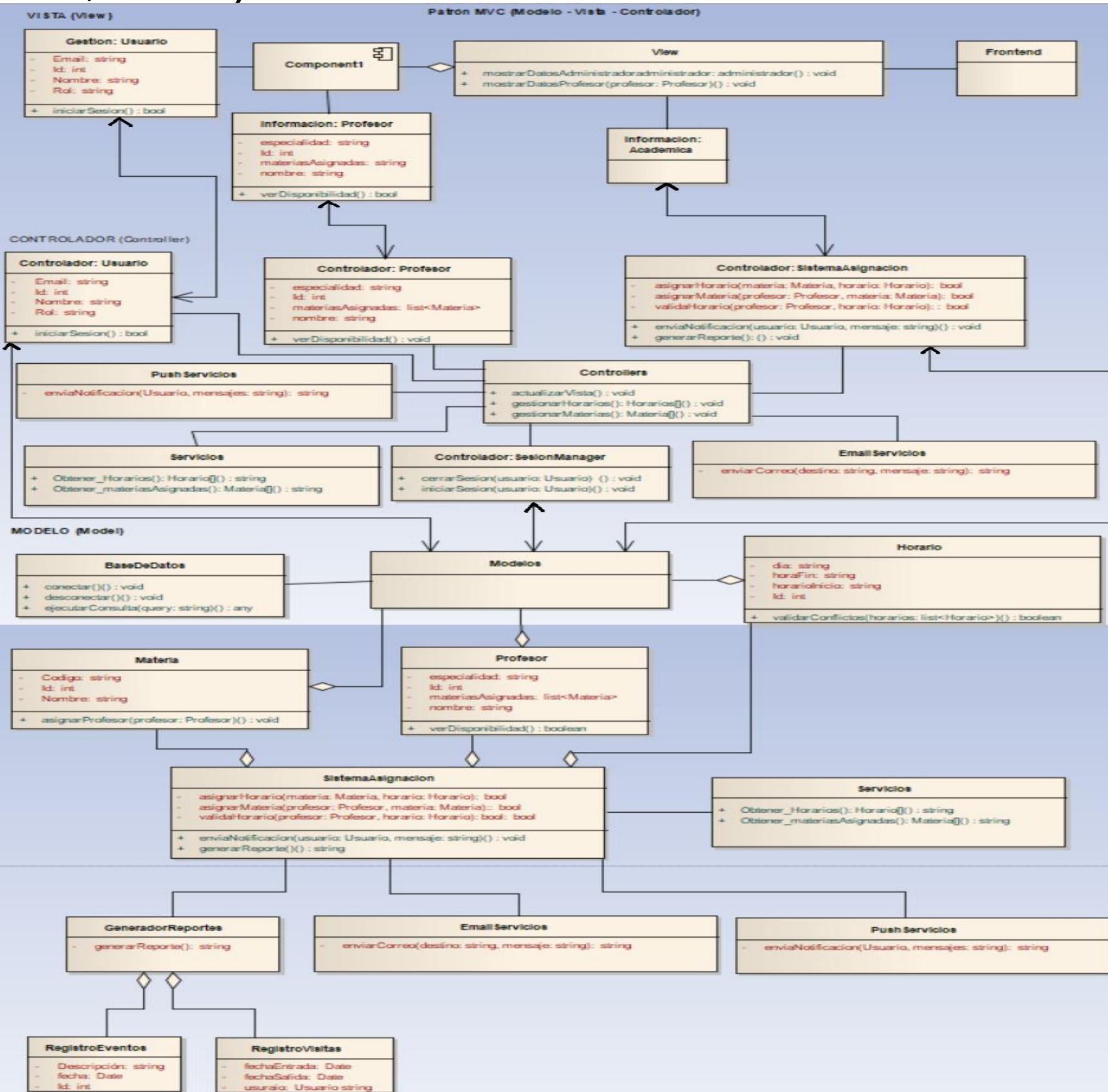


Figura 2: Patrón Modelo-Vista-Controlador (MVC):.

Lenguaje de Definición de Interfaces (IDL)

Este sistema está diseñado con arquitectura web basada en el modelo cliente-servidor, por lo que las interfaces entre los componentes siguen estándares modernos como lo son:

- RESTful API: para comunicación entre el frontend web y el backend (servidor Django).
- ORM de Django: actúa como interfaz hacia la base de datos MySQL.
- JSON: formato de datos para intercambios entre frontend y backend.
- HTML/CSS/JavaScript: lenguajes para la interfaz visual de usuario.

Cada una de estas interfaces expone funciones específicas que están alineadas con los casos de uso y requisitos funcionales (RF) del sistema.

El sistema de asignación de horarios utiliza un enfoque basado en Lenguajes de Definición de Interfaz (IDL, por sus siglas en inglés) para estandarizar la comunicación entre los componentes del sistema, así como su interacción con sistemas externos. Un IDL permite especificar de forma clara y estructurada las operaciones que están disponibles en cada módulo o componente, facilitando así la interoperabilidad, independientemente del lenguaje de programación subyacente.

Cada componente funcional del sistema tiene definida una interfaz clara y separada que especifica las operaciones que puede realizar, y cómo interactúa tanto con los actores del sistema como con otros servicios o recursos externos, tales como la base de datos MySQL y el servidor backend (Django). Estas interfaces son fundamentales para permitir la implementación modular y escalable del sistema.

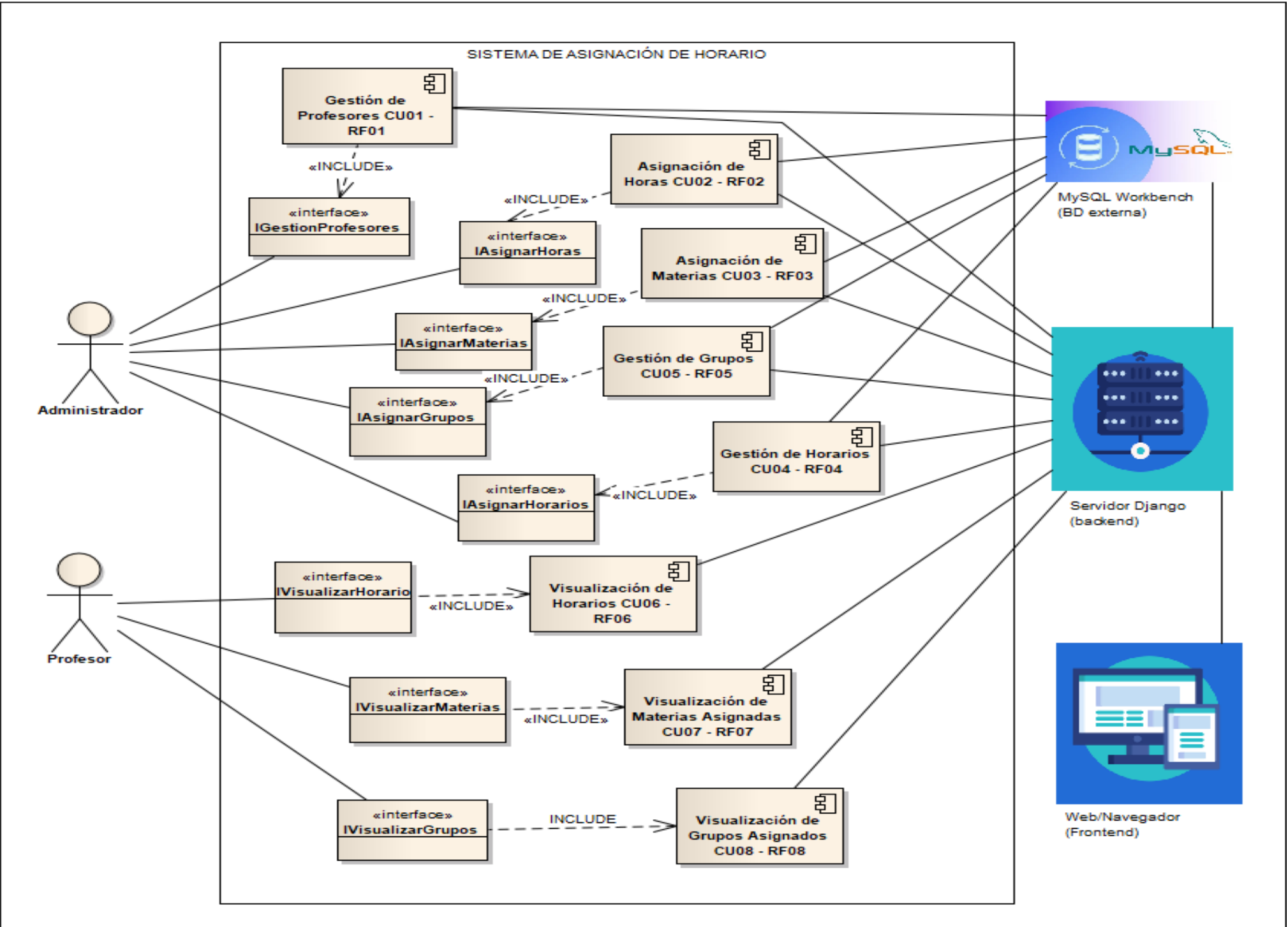


Figura 3: Lenguajes de definición de interfaz (IDL), diagrama de componentes UML Interacción (5.10)

DIAGRAMA DE SECUENCIA 1 DEL CASO DE USO 01 AGREGAR PROFESOR:

El diagrama de secuencia ilustra el flujo de interacciones entre los diferentes componentes del sistema durante el proceso de agregar un nuevo profesor. El proceso se inicia cuando el Administrador, a través de la interfaz de usuario, selecciona la opción para iniciar la gestión de profesores. La interfaz muestra una pantalla de bienvenida con varias opciones, incluyendo la de agregar un nuevo profesor, la cual el Administrador selecciona. Al seleccionar la opción de agregar profesor, la interfaz solicita al Administrador que llene un formulario con los datos del nuevo profesor, como el ID de empleado, nombre, correo electrónico, especialidad y materias asignadas. A medida que el Administrador ingresa cada dato, la interfaz envía mensajes al componente "Controllers" para validar la información ingresada. El "Controllers" a su vez interactúa con el "SesionManager" y la "Base Datos" para realizar las validaciones necesarias, como verificar si ya existe un profesor con el mismo ID, nombre o correo electrónico, y para confirmar la existencia y disponibilidad de la especialidad y las materias asignadas.

Una vez que todos los datos han sido ingresados y validados con éxito, el Administrador selecciona el botón "Registrar". La interfaz envía un mensaje "RegistrarProfesor()" al "Controllers", el cual inicia el proceso de guardar los datos del nuevo profesor en la "Base Datos" mediante el mensaje "GuardarDatosProfesor()". La "Base Datos" confirma que el nuevo profesor ha sido agregado. Finalmente, el "Controllers" informa a la interfaz de usuario mediante el mensaje "MostrarConfirmacionRegistroProfesor()" que el registro del profesor ha sido exitoso, mostrando un mensaje de confirmación al Administrador. Adicionalmente, se registra un evento de "Nuevo UsuarioProfesor" en el componente "Registro Eventos". En resumen, el diagrama de secuencia detalla cómo la interfaz de usuario, el "Controllers", el "SesionManager", la "Base Datos" y el "Registro Eventos" colaboran para llevar a cabo la adición de un nuevo profesor al sistema, incluyendo la validación de los datos ingresados y la confirmación de la operación al Administrador.

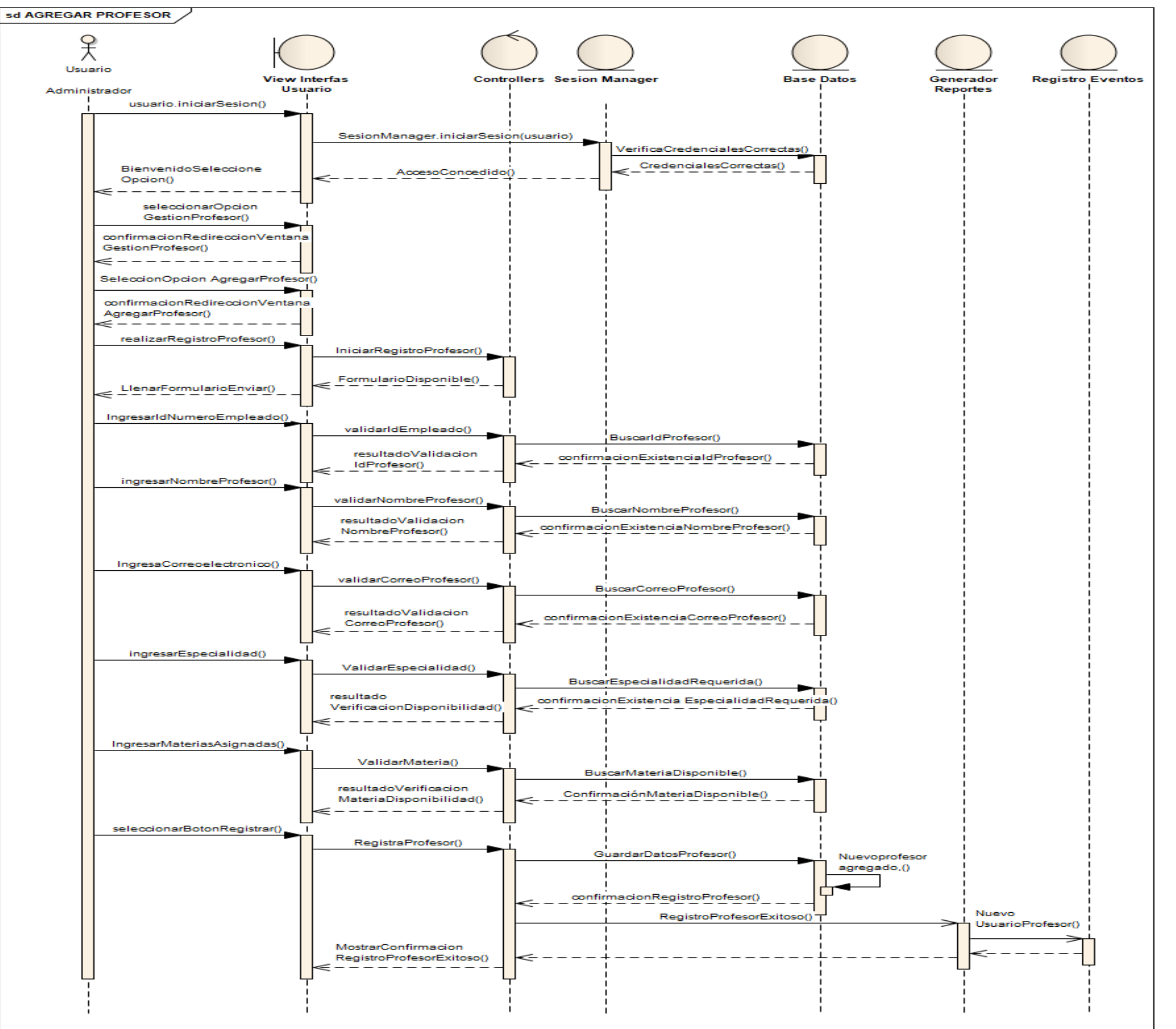
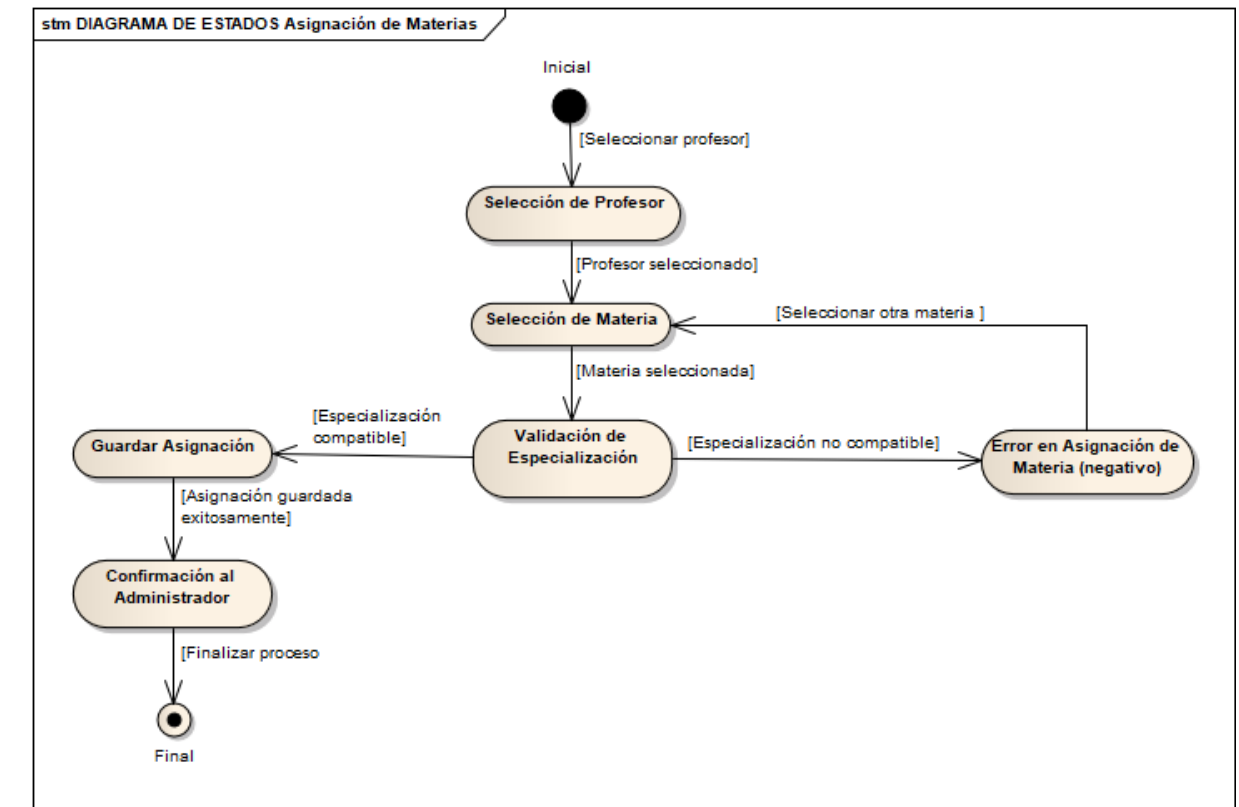


Figura 4: DIAGRAMA DE SECUENCIA DEL CASO DE USO 01 AGREGAR PROFESOR

Dinámica de estados y Lógica procedimental algorítmica

Dinámica de estados (5.11). La Dinámica de Estados detalla cómo los elementos clave del sistema, como los profesores, las materias o los horarios, evolucionan a través de diferentes fases a lo largo del tiempo.

Figura 5: DIAGRAMA (5) ESTADOS ASIGNAR MATERIAS A PROFESOR



Lógica procedimental algorítmica (5.12)

Esta sección del Documento de Diseño de Software se centra en detallar la lógica y los procedimientos paso a paso que el sistema implementará para llevar a cabo sus diversas funcionalidades. Va más allá de las descripciones generales y profundiza en los algoritmos específicos que se utilizarán.

Figura 5: Algoritmo 7: SubProceso AsignarGrupos()

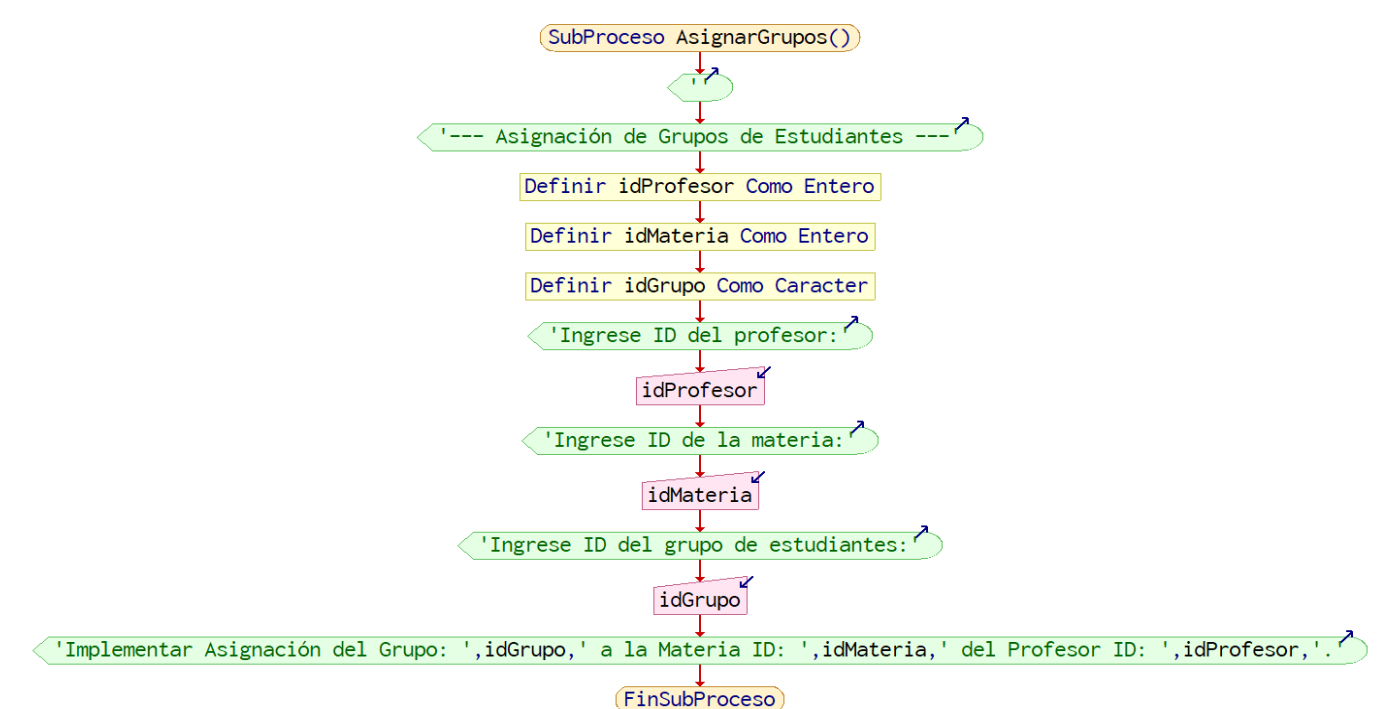


Figura 5: Funcionamiento del Algoritmo de Gale-Shapley para la Asignación de Materias (CU03):



Conclusiones

El algoritmo Gale-Shapley. Para la asignación de Horario para profesores implementa una solución eficiente y estable para el problema de asignación entre profesores y materias, considerando las preferencias mutuas de ambos. A lo largo del proceso, cada profesor propone a las materias según su orden de preferencia, y cada materia acepta temporalmente al profesor más preferido disponible, rechazando al anterior si se encuentra una mejor opción. Este ciclo se repite hasta que no queden profesores sin asignar o sin opciones de preferencia restantes.